

Проектирование микропроцессорной системы на базе МК i8051

Описание программной части

Листов 27

Выполнил: Иванов Д.А.

Группа: А-12-23

Вариант: 5

Проверил: Михалин С.Н.

Инд. № подл.	Подп. и дата
Взам. инд. №	Инд. № дубл.
Подп. и дата	Подп. и дата

Москва, 2026

АННОТАЦИЯ

В данном документе приведено описание программы «Программа управления объектом», предназначенной для управления объектом микропроцессорной системой (МПС) на базе микроконтроллера AT89S53: ввода данных с клавиатуры и от внешнего устройства, формирования управляющих сигналов Y_1 и Y_2 и вывода информации на семисегментный индикатор.

Для функционирования данной программы необходима МПС, собранная по схеме электрической принципиальной варианта 5 (пояснительная записка по аппаратной части).

Исходным языком программы «Программа управления объектом» является ассемблер микроконтроллеров семейства MCS-51.

Основными функциями программы являются: инициализация МПС, сканирование клавиатуры, работа с кольцевым буфером на 1654 байт, выработка stroba Y_1 длительностью 241 мс, выработка сигнала Y_2 со стробом 2062 мкс, индикация на одном разряде.

Оформление программного документа «Описание программы» произведено по требованиям ЕСПД (ГОСТ 19.101-77, ГОСТ 19.103-77, ГОСТ 19.104-78, ГОСТ 19.105-78, ГОСТ 19.106-78, ГОСТ 19.402-78, ГОСТ 19.604-78).

СОДЕРЖАНИЕ

1. Общие сведения.....	4
2. Функциональное назначение.....	5
2.1. Классы решаемых задач.....	5
2.2. Назначение программы.....	5
2.3. Сведения о функциональных ограничениях на применение.....	5
3. Описание логической структуры.....	6
3.1. Алгоритм программы.....	6
3.2. Структура программы с описанием функций составных частей и связи между ними.....	17
3.3. Связи программы с другими программами.....	18
4. Входные данные.....	19
4.1. Клавиатура.....	19
4.2. Сигнал X2.....	19
5. Выходные данные.....	20
5.1. Характер и организация выходных данных.....	20
5.2. Формат, описание и способ кодирования выходных данных.....	20
ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ.....	22
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ.....	23
ПРИЛОЖЕНИЕ А. Листинги программ.....	24

1. ОБЩИЕ СВЕДЕНИЯ

Обозначение и наименование программы — «Программа управления объектом», вариант 5, группа А-12-23.

Программа низкого уровня не требует для своей работы иных программ. Исходным языком программирования является ассемблер MCS-51 (синтаксис Intel ASM51, совместимый с Keil A51 и MCU 8051 IDE).

Исходные тексты набирались на языке ассемблера 8051 и проверялись утилитой `mpscode` из репозитория проекта. Она использует ассемблер `asm51` и эмулятор `emu51`, выполняет программы на модели устройств моей схемы и дополнительно проверяет правила оформления для работа кафедры. Синтаксис исходных файлов совместим с Keil A51.

2. ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ

2.1. Классы решаемых задач

- инициализация МПС;
- сканирование клавиатуры — ввод сигнала X_1 (число, введённое с клавиатуры);
- формирование сигнала Y_1 (строб заданной длительности);
- ввод сигнала X_2 (сохранение данных в кольцевом буфере);
- формирование сигнала Y_2 и сопровождающего строба;
- вывод символа на индикатор.

2.2. Назначение программы

МПС принимает информацию $\{X\}$ об объекте управления от датчиков, вырабатывает управляющие воздействия $\{Y\}$ в соответствии с заданным алгоритмом управления и подаёт их на исполнительные механизмы. Программа разбита на подзадачи по функциональному признаку:

- инициализация МПС;
- обработка прерываний: опрос клавиатуры, запись в буфер, таймеры генераторов сигналов Y ;
- определение кода нажатой клавиши;
- вывод данных на семисегментный индикатор;
- формирование сигнала Y_2 ;
- обеспечение работы кольцевого буфера (запись и чтение отсчётов X_2).

В соответствии с вариантом (табл. 1 ТЗ) разработан код трёх программ:

- программа 1 — обработчик прерывания клавиатуры (сканирование матрицы), код нажатой клавиши в аккумуляторе или 0FFh;
- программа 2 — запись в кольцевой буфер (данные через аккумулятор) с управлением флагами пустоты и переполнения;
- программа 3 — расчёт и выдача сигнала Y_2 со стробом длительностью T_2 (прерывание таймера).

2.3. Сведения о функциональных ограничениях на применение

В качестве основы для построения МПС выбран микроконтроллер семейства MCS-51 — AT89S53, тактовая частота 12 МГц (машинный цикл 1 мкс). Таймер 2 и сторожевой таймер не используются (совместимость с i8051).

Интерфейсы:

- входные данные X_2 — 8-разрядное целое без знака по независимой параллельной шине со стробом записи низкого уровня в двухпортовое ОЗУ, нерегулярно, до 4 отсчётов в секунду;
- входные данные X_1 — десятичная цифра 0–9 с матричной клавиатуры 3×4 (сканирование по столбцам);
- выходной сигнал Y_2 — 8-разрядный параллельный код в выходном регистре со стробом низкого уровня длительностью 2062 мкс на выводе P1.6;
- выходной сигнал Y_1 — строб низкого уровня длительностью 241 мс на выводе P1.5;
- индикация — один семисегментный индикатор (общий анод), время индикации символа не более $T_3 = 2,590$ с.

3. ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ

3.1. Алгоритм программы

После сброса программа устанавливает $SP = 07h$, разрешает дешифратор сигналом CS_EN , задаёт начальные состояния буфера, индикатора и стробов, а затем настраивает таймеры и прерывания (рисунок 1). В основном цикле проверяются $X1$, состояние кольцевого буфера и занятость строба $Y2$: если обработка разрешена, очередной отсчёт читается из буфера, по нему рассчитывается $Y2$ и результат передаётся во внешний регистр; при $X1 = 0$ буфер не читается (рисунок 2).

Асинхронные события обрабатываются отдельно. $INT0$ запускает опрос клавиатуры и реакцию на найденную клавишу (рисунки 3 и 4), а $INT1$ переносит принятый через правый порт IDT7005 отсчёт $X2$ из ячейки обмена в кольцо (рисунок 6). $Timer0$ завершает выдержку $Y1$ или время индикации $T3$, а $Timer1$ возвращает строб $Y2$ в пассивное состояние. В трёх программах варианта отдельно реализованы и проверены процедуры опроса клавиатуры, записи в буфер и расчёта $Y2$ со стробом.

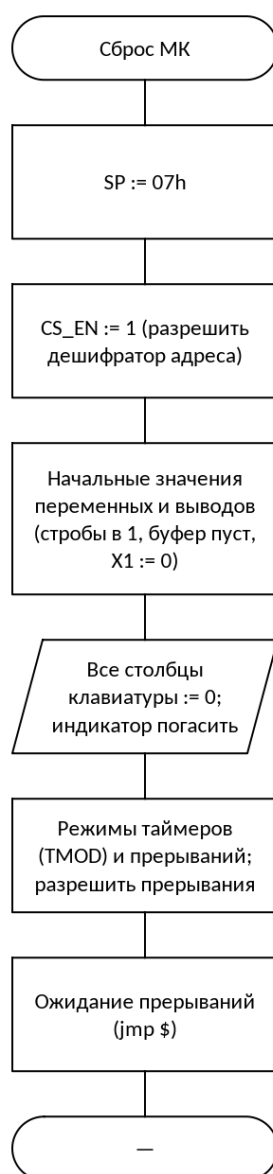


Рис. 1. Схема алгоритма: Инициализация МПС и основная программа

Сначала указатель стека устанавливается в $07h$, чтобы стек рос из области после банка регистров 0 и не пересекался с переменными. Затем включается дешифратор адресов, буфер

помечается пустым, X1 обнуляется, а оба строба переводятся в пассивную единицу. Все столбцы клавиатуры опускаются в ноль, чтобы нажатие гарантированно сформировало спад на INT0, индикатор гасится. После настройки таймеров и разрешения прерываний основная программа переходит к ожиданию событий.

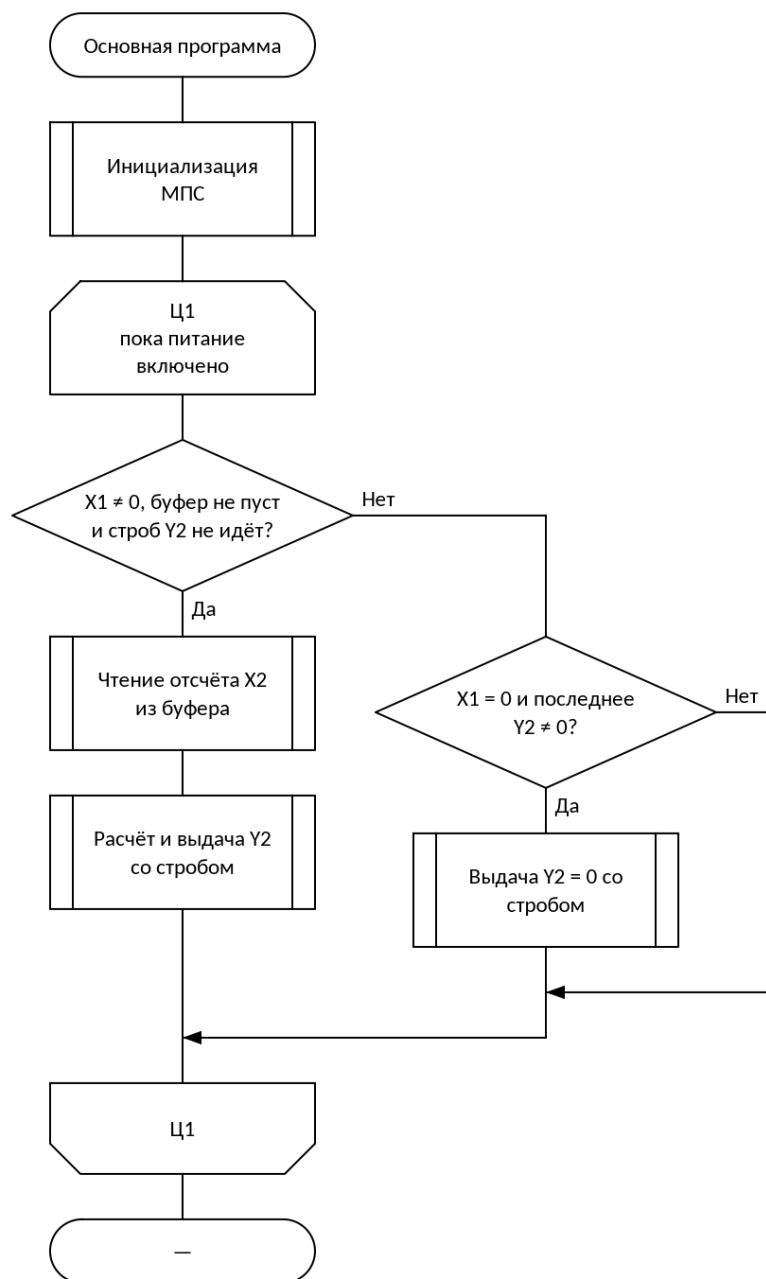


Рис. 2. Схема алгоритма: Основной цикл программы

Основной цикл сначала проверяет, что X1 не равен нулю, буфер содержит данные и предыдущий строб Y2 уже завершён. При выполнении этих условий из кольца извлекается один отсчёт X2, рассчитывается новое значение Y2 и запускается его строб. Когда $X1 = 0$, чтение буфера запрещено требованием ТЗ; если в регистре ещё осталось активное значение, в него один раз записывается ноль со стробом. В остальных случаях цикл ничего не меняет и повторяет проверку.

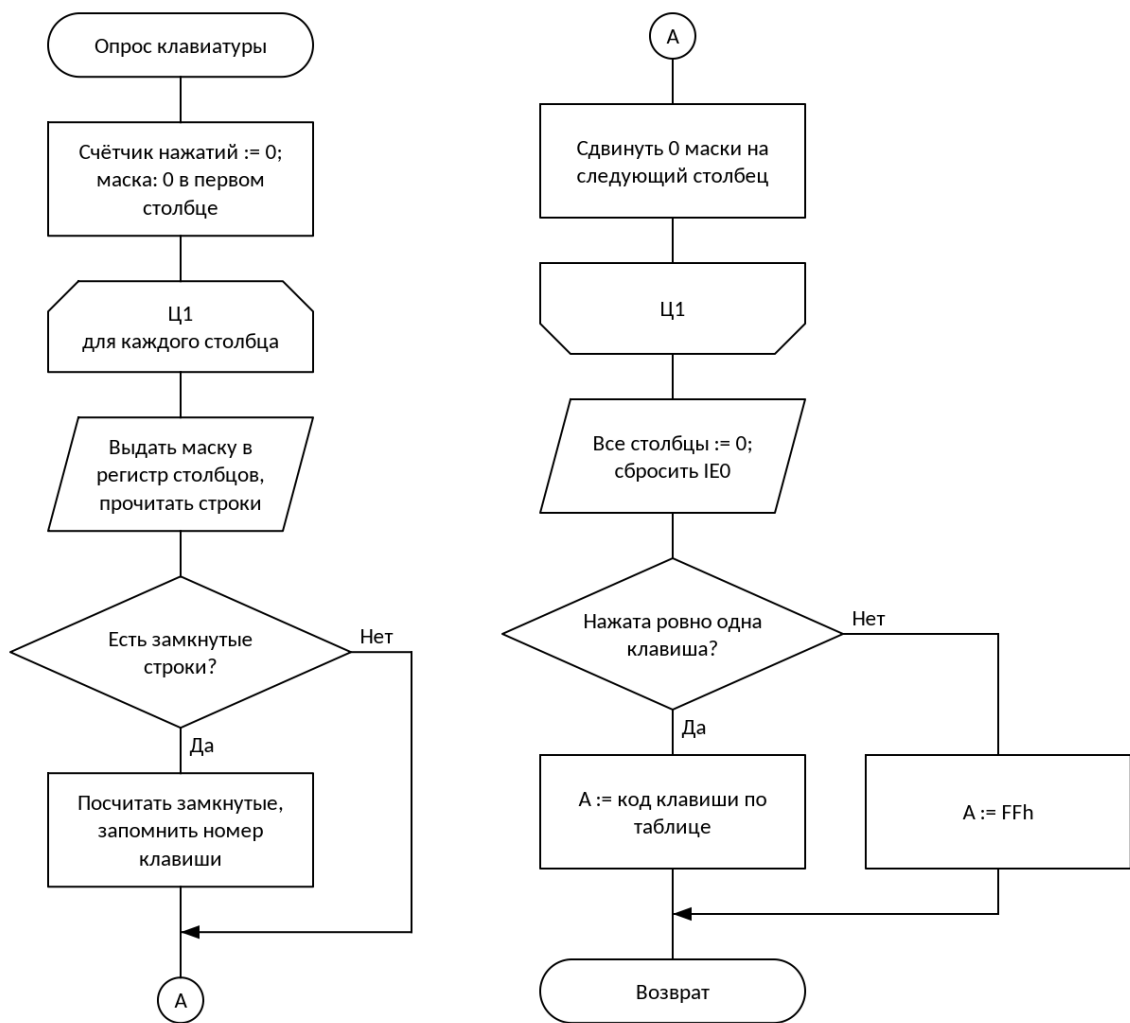


Рис. 3. Схема алгоритма: Опрос клавиатуры (обработчик INT0)

Процедура по очереди выбирает каждый из трёх столбцов клавиатуры нулевым уровнем и читает четыре младших бита строк. Для каждого обнаруженного замыкания увеличивается общий счётчик, а код первого контакта вычисляется по расположению клавиш 3×4. После полного прохода все столбцы снова записываются нулями, иначе следующее нажатие не создаст запрос **INT0**; дополнительно сбрасывается **IE0**. Код возвращается только при одном замкнутом контакте, а при отсутствии клавиши или нескольких нажатиях выдаётся **0FFh**.

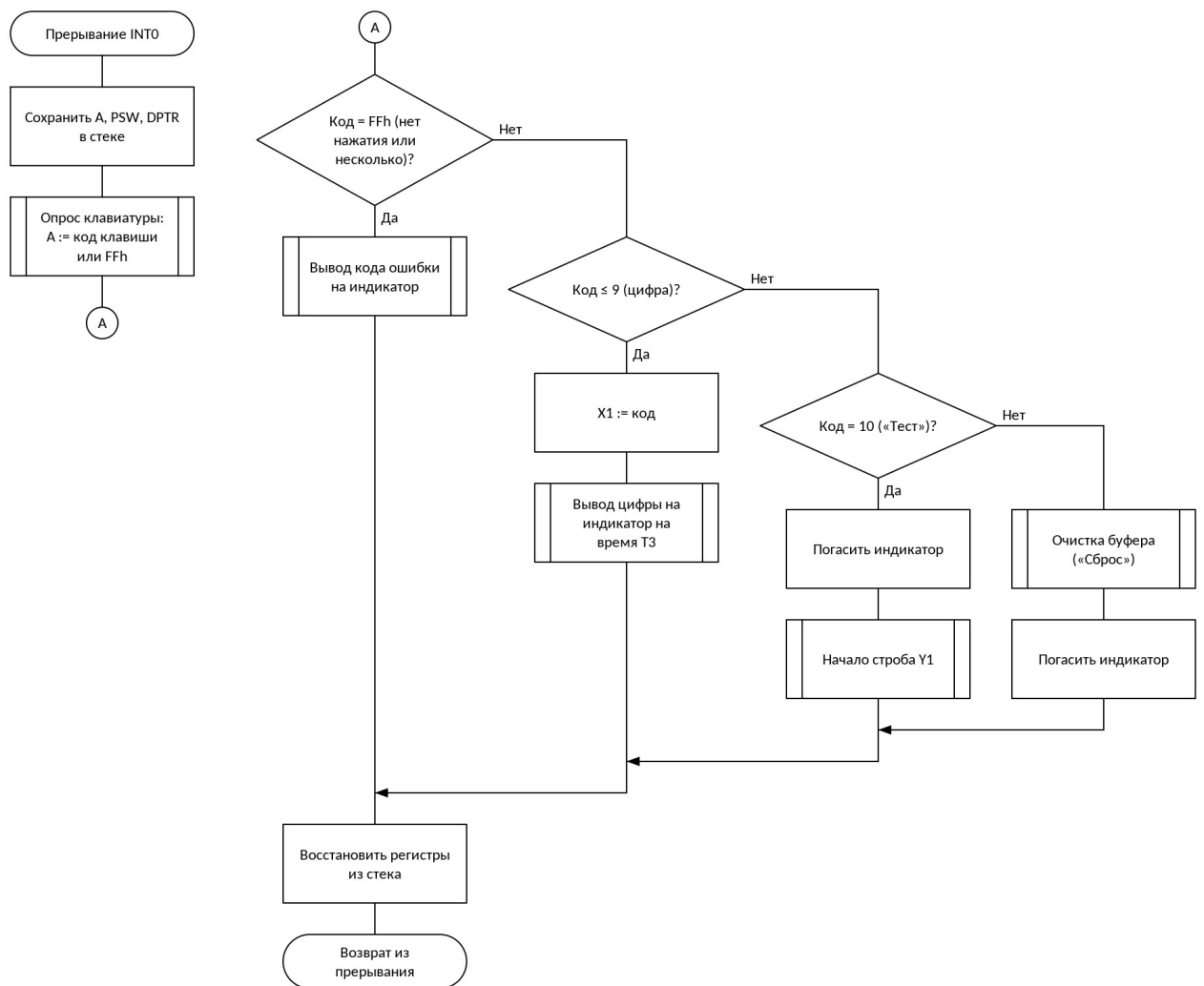


Рис. 4. Схема алгоритма: Обработчик прерывания INTO: реакция на клавишу

Обработчик сохраняет используемые регистры и вызывает процедуру сканирования клавиатуры. Цифровой код от 0 до 9 записывается в X1 и выводится на индикатор на время T3. Клавиша «Тест» гасит индикатор и запускает строб Y1, а «Сброс» очищает кольцевой буфер и также выключает индикацию. Значение 0FFh считается ошибочным опросом; после выбранной реакции контекст восстанавливается и выполняется возврат из прерывания.



Рис. 5. Схема алгоритма: Запись отсчёта в кольцевой буфер

Перед записью программа снимает старый флаг переполнения и проверяет состояние указателей. В принятой политике используются все 1654 ячейки: совпадение «головы» и «хвоста» означает полный буфер только при сброшенном флаге пустоты. В этом случае «хвост» передвигается, старейший отсчёт теряется и устанавливается F_OVF. Новый байт записывается по адресу «головы», после чего она сдвигается с заворотом от 0A675h к 0A000h, а F_EMPTY сбрасывается.



Рис. 6. Схема алгоритма: Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства

Спад внешнего строба вызывает INT1, после чего обработчик сохраняет контекст. Считывать данные сразу нельзя: правый порт IDT7005 завершает запись по фронту строба, поэтому сначала ожидается возвращение линии P3.3 в единицу. Затем МК через левый порт читает байт из ячейки обмена 0BFFFh и вызывает процедуру записи в кольцевой буфер. После восстановления регистров обработчик завершается командой RETI.

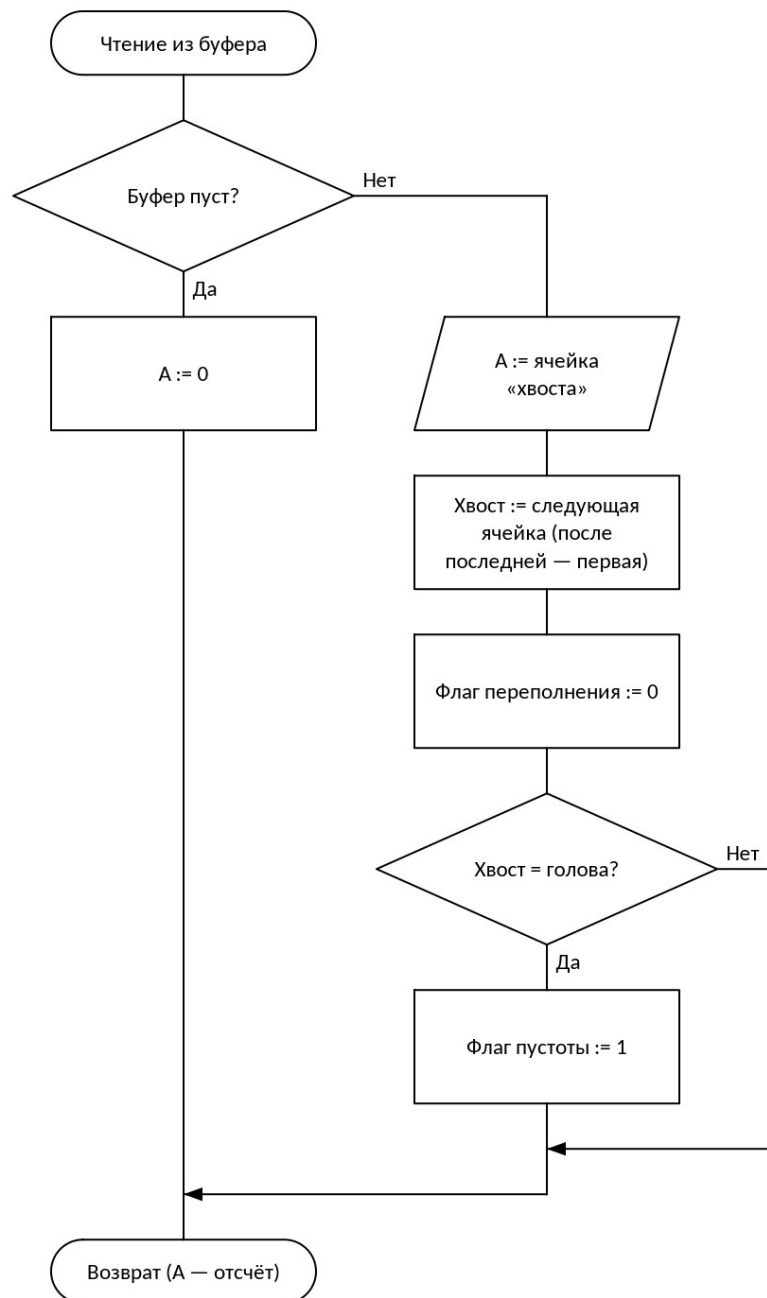


Рис. 7. Схема алгоритма: Чтение отсчёта из кольцевого буфера

Если установлен флаг пустоты, процедура возвращает ноль и не меняет указатели. Иначе байт читается из ячейки, на которую указывает «хвост», после чего указатель переходит к следующему адресу с заворотом в начало кольца. Успешное чтение снимает признак переполнения. Когда новый «хвост» догоняет «голову», прочитан последний отсчёт и устанавливается F_EMPTY.

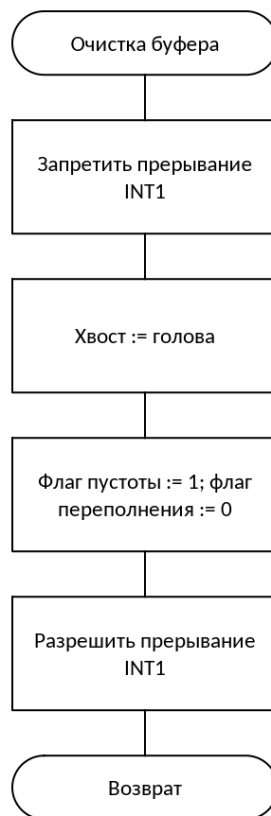


Рис. 8. Схема алгоритма: Очистка кольцевого буфера (клавиша «Сброс»)

На время очистки запрещается INT1, чтобы новый отсчёт не изменил буфер между операциями с указателями. «Хвост» приравнивается к «голове»: содержимое памяти физически не стирается, но все старые байты становятся недоступными для чтения. Одновременно устанавливается флаг пустоты и снимается флаг переполнения. После получения согласованного состояния приём X2 снова разрешается.

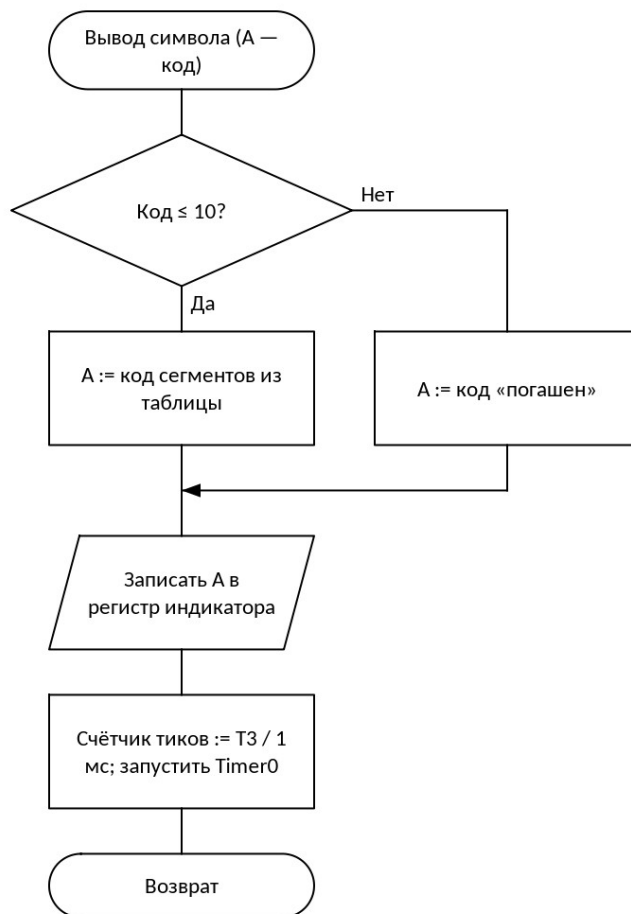


Рис. 9. Схема алгоритма: Вывод символа на индикатор

Для кодов 0–10 выбирается соответствующая комбинация сегментов цифр или символа Е; остальные значения преобразуются в код погашенного индикатора. Так как в моём варианте используется общий анод, нулевой бит включает сегмент, а 0FFh выключает все сегменты. Подготовленный байт записывается в регистр индикатора. Затем Timer0 запускается на 2590 тиков по 1 мс, чтобы символ автоматически погас через T3.

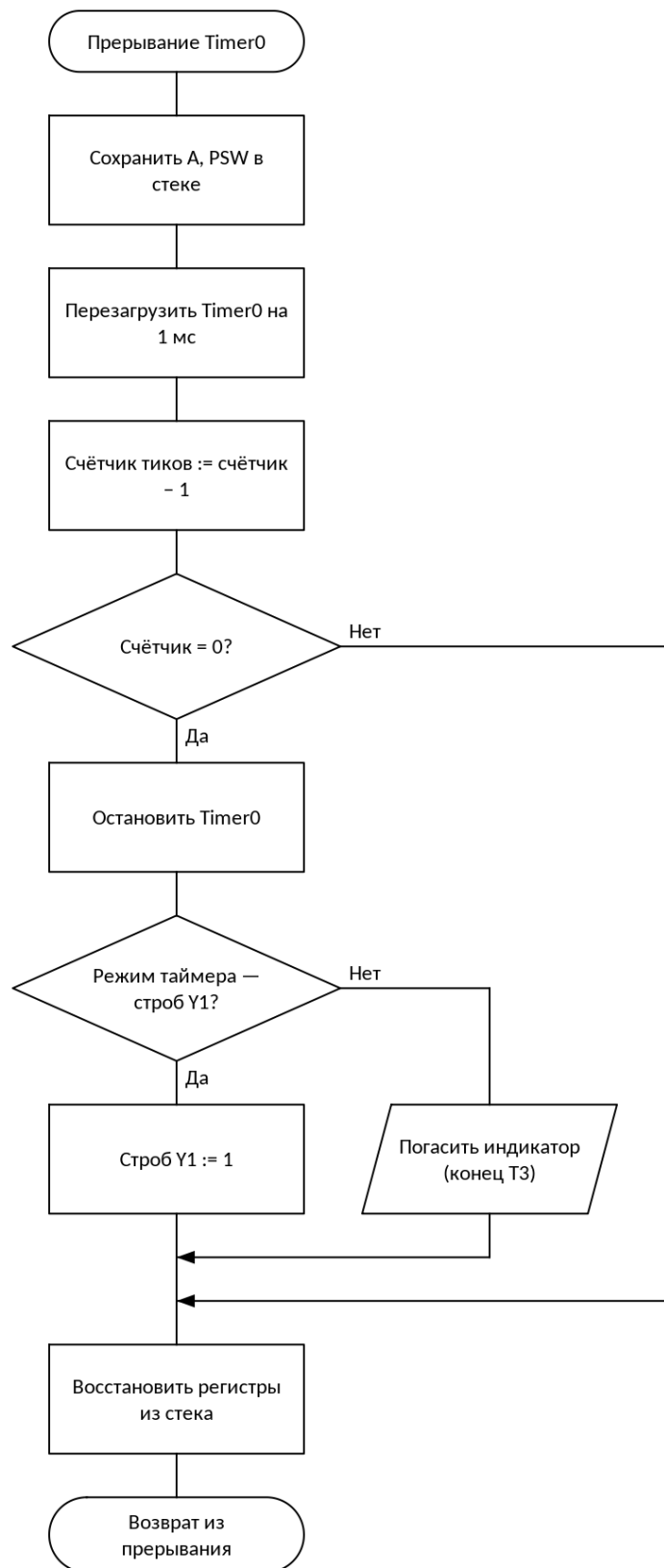


Рис. 10. Схема алгоритма: Обработчик прерывания Timer0: отсчёт T1 и T3

При каждом переполнении Timer0 сохраняются аккумулятор и PSW, а регистр таймера снова загружается значением для тика 1 мс. Программный счётчик уменьшается на единицу. Пока он не обнулится, обработчик сразу возвращается; при нуле таймер останавливается. Дальнейшее действие определяется режимом: для T1 линия Y1 возвращается в единицу, а для T3 гасится индикатор, после чего контекст восстанавливается.



Рис. 11. Схема алгоритма: Начало строба Y1

Для длительности T1 в счётчик заносится 241 тик, а Timer0 загружается значением 0FC18h, соответствующим 1 мс. Линия P1.5 переводится в активный ноль и таймер запускается. Сама процедура сразу возвращается и не ждёт 241 мс в цикле, поэтому МК может обслуживать другие события. Завершение строба выполняет обработчик Timer0.

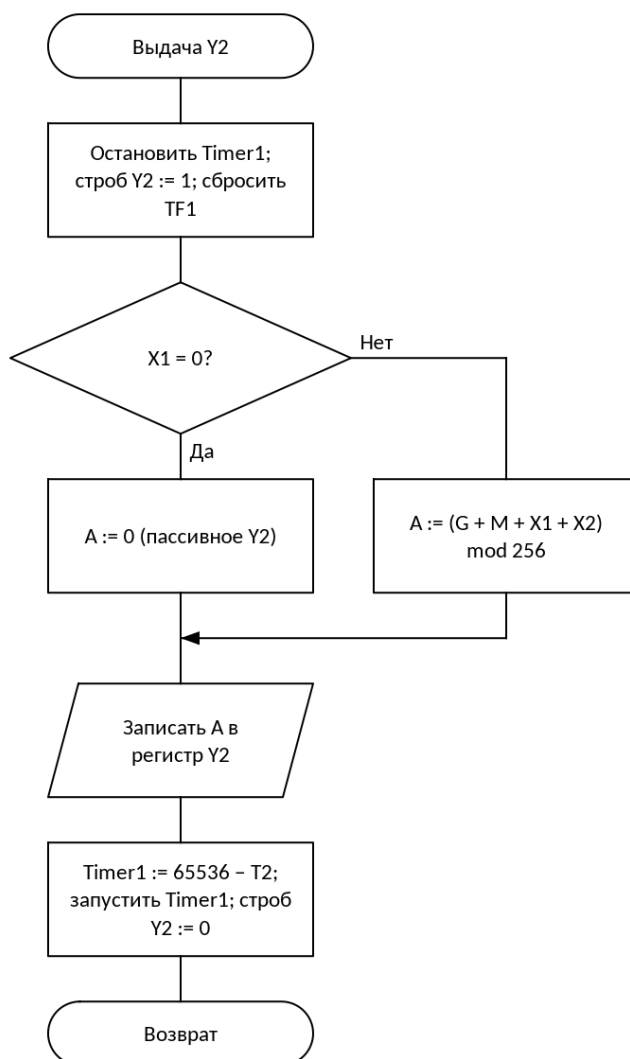


Рис. 12. Схема алгоритма: Расчёт и выдача сигнала Y2

При $X1 = 0$ в аккумулятор помещается пассивное значение $Y2$, равное нулю. В остальных случаях рассчитывается $(17 + X1 + X2) \bmod 256$, причём перенос за пределы байта отбрасывается. Сначала результат записывается по адресу $0E000h$ в выходной регистр, чтобы к началу строга данные уже были устойчивы. Затем Timer1 загружается значением $0F7F2h$, линия $P1.6$ опускается в ноль и начинается отсчёт 2062 мкс.

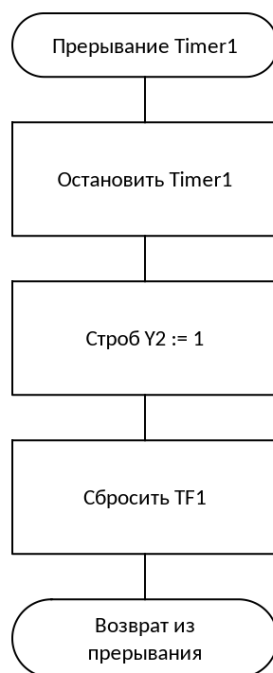


Рис. 13. Схема алгоритма: Обработчик прерывания Timer1: конец строга $Y2$

Переполнение Timer1 означает, что требуемые 2062 мкс прошли. Обработчик останавливает таймер, возвращает $P1.6$ в пассивную единицу и снимает флаг TF1. Он не использует аккумулятор и PSW, поэтому не меняет вычисления основной программы. Возврат выполняется через RETI, после чего можно формировать следующий строб.

3.2. Структура программы с описанием функций составных частей и связи между ними

Программа состоит из инициализации, основной программы (ожидание прерываний) и подпрограмм, описанных ниже. Описание каждой подпрограммы взято из заголовочного комментария в исходном тексте (приложение А).

3.2.1 Подпрограмма KbScan (программа 1)

Назначение: опрос всех контактов матрицы 3×4 по столбцам.

Входные данные: нет.

Выходные данные: A — код единственной клавиши по таблице 3 T3, иначе 0FFh.

Используемые регистры: B, DPTR, R0–R5, PSW. Перед возвратом снова опускает все столбцы.

3.2.2 Подпрограмма BufWrite (программа 2)

Назначение: запись байта в кольцевой буфер с политикой полной ёмкости V.

Входные данные: A — новый отсчёт.

Выходные данные: F_EMPTY = 0; F_OVF = 1 только при вытеснении старого байта.

Используемые регистры: A, B, DPTR, R6, R7, PSW.

3.2.3 Подпрограмма Y2Out (программа 3)

Назначение: вычисление $Y2$ и запуск сопровождающего строга Timer1.

Входные данные: $X1$ и $X2$ в ОЗУ.

Выходные данные: байт записан в ADR_Y2, PIN_Y2 удерживается в нуле T2 мкс.

Используемые регистры: A, DPTR, PSW.

3.2.4 Обработчик прерывания Timer1 (программа 3)

Назначение: Timer1 — окончание строба Y2.

Распределение ресурсов микроконтроллера — прерывания, адреса внешних устройств и переменные — приведено в таблицах 1–3.

Таблица 1 — Прерывания

Источник	Вектор	Назначение	Запуск
INT0 (P3.2)	0003h	нажатие клавиши	по спаду
Timer0	000Bh	отсчёт T_1 (241 мс) и T_3 (2590 мс), тик 1 мс	переполнение
INT1 (P3.3)	0013h	строб записи X_2	по спаду
Timer1	001Bh	отсчёт T_2 (2062 мкс)	переполнение
UART	0023h	не используется (заглушка)	—

Таблица 2 — Адреса внешних устройств (обращение командами movx при P3.4 = 1)

Устройство	Выход дешифратора	Адреса
Индикатор	Y6	0C000h–0DFFFh
Клавиатура	Y1	2000h–3FFFh
Буфер (IDT7005)	Y5	0A000h–0BFFFh
Регистр Y2	Y7	0E000h–0FFFFh

Таблица 3 — Переменные во внутреннем ОЗУ

Имя	Адрес	Назначение
HEAD_L, HEAD_H	47h, 48h	«голова» — адрес ячейки буфера для следующей записи
TAIL_L, TAIL_H	49h, 4Ah	«хвост» — адрес ячейки буфера для следующего чтения
F_EMPTY	20h.5 (бит 05h)	флаг пустоты буфера
F_OVF	22h.1 (бит 11h)	флаг переполнения буфера
X1	30h	сигнал X_1 — цифра с клавиатуры
X2	31h	сигнал X_2 — отсчёт из буфера

3.3. Связи программы с другими программами

Программа «Программа управления объектом» в ходе своей работы не использует других программ.

4. ВХОДНЫЕ ДАННЫЕ

4.1. Клавиатура

Клавиатура — матрица 3×4 (3 столбцов × 4 строк), сканирование по столбцам: в регистр столбцов (адрес 2000h, разряды D0...D2) выдаётся 0 в опрашиваемый столбец, строки читаются через повторитель по тому же адресу (разряды D0...D3, 0 — контакт замкнут). Нажатие любой клавиши при всех столбцах в 0 вызывает прерывание INT0. Коды клавиш (табл. 3 ТЗ) и их расположение — в таблице 4.

Таблица 4 — Расположение клавиш и их коды («клавиша» — код)

	Col1	Col2	Col3
Row1	«1» — 1	«2» — 2	«3» — 3
Row2	«4» — 4	«5» — 5	«6» — 6
Row3	«7» — 7	«8» — 8	«9» — 9
Row4	«0» — 0	«Т» — 10	«С» — 11

Результат опроса — код единственной нажатой клавиши (0–9 — цифры, 10 — «Тест», 11 — «Сброс») или 0FFh, если не нажато ничего или нажато больше одной клавиши. Цифра становится значением X_1 (начальное значение $X_1 = 0$).

4.2. Сигнал X2

X_2 — 8-разрядное целое без знака. Внешнее устройство записывает отсчёт в двухпортовое ОЗУ IDT7005 через свой порт и сопровождает запись стробом, который вызывает прерывание INT1. Отсчёты хранятся в кольцевом буфере на 1654 байт (адреса 0A000h–0A675h).

5. ВЫХОДНЫЕ ДАННЫЕ

5.1. Характер и организация выходных данных

Индикация. На индикаторе отображается последняя введенная цифра — до ввода следующей, но не дольше $T_3 = 2,590$ с; при ошибке — код ошибки с горящей точкой др. Начальное состояние — индикатор погашен.

Кольцевой буфер во внешнем ОЗУ. Ячейки 0A000h–0A675h. «Голова» — адрес ячейки для следующей записи, «хвост» — для следующего чтения; после последней ячейки следует первая. Буфер пуст — флаг пустоты = 1 (голова = хвост). Если буфер заполнен, новый отсчет затирает самый старый (хвост сдвигается) и устанавливается флаг переполнения; после чтения флаг переполнения сбрасывается.

Управляющие сигналы. Y_1 — строб низкого уровня длительностью 241 мс по клавише «Тест». $Y_2 = (12 + 5 + X_1 + X_2) \bmod 256$ при $X_1 \neq 0$, иначе $Y_2 = 0$ (пассивное состояние); вырабатывается, пока в буфере есть данные и $X_1 \neq 0$. Каждая запись Y_2 в выходной регистр сопровождается стробом длительностью 2062 мкс; до записи регистр выдает предыдущее значение.

Расчётные параметры работы программы. Константы таймеров (кварц 12 МГц, машинный цикл $T_{\text{мц}} = 12 / f = 1$ мкс, режим 1 — 16 бит):

ТН1:ТЛ1 = 65536 – $T_2 / T_{\text{мц}} = 65536 - 2062 = 63474 = 0F7F2h$ (строб Y_2 , таймер 1)

ТН0:ТЛ0 = 65536 – 1 мс / $T_{\text{мц}} = 65536 - 1000 = 64536 = 0FC18h$ (тик 1 мс, таймер 0)

$N_{T1} = T_1 / 1 \text{ мс} = 241$; $N_{T3} = T_3 / 1 \text{ мс} = 2590$ (число тиков)

Минимальное время заполнения буфера (при $X_1 = 0$ сигнал Y_2 пассивен и буфер не читается, отсчеты поступают с максимальной интенсивностью):

$t_{\text{min}} = V / f_{X2} = 1654 / 4 = 413,5$ с (6,9 мин).

Максимальная глубина стека для программы 1 — 4 байта (SP от 07h до 0Bh). Значение получено прогоном программы в симуляторе на всех сценариях проверки (каждая клавиша, одновременные нажатия, помеха на INT0) и складывается из адреса возврата при входе в обработчик INT0 (2 байта), адреса возврата при вызове процедуры опроса (2 байта) и сохраняемых в стеке регистров.

5.2. Формат, описание и способ кодирования выходных данных

Код символа (табл. 2 ТЗ) преобразуется в код сегментов по таблице 5: разряды D0...D7 регистра индикатора соответствуют сегментам a, b, c, d, e, f, g, dr; индикатор с общим анодом — сегмент горит при 0.

Таблица 5 — Коды сегментов индикатора (dr g f e d c b a)

Код	Символ	Двоичный	Шестн.
0	«0»	11000000	0C0h
1	«1»	11111001	0F9h
2	«2»	10100100	0A4h
3	«3»	10110000	0B0h
4	«4»	10011001	99h
5	«5»	10010010	92h
6	«6»	10000010	82h
7	«7»	11111000	0F8h
8	«8»	10000000	80h
9	«9»	10010000	90h
10	«Е»	10000110	86h
11 и больше	погашен	11111111	0FFh

Таблица 6 — Коды ошибок (символ с горящей точкой dp, индикация до нажатия любой клавиши, но не дольше T3)

Индикация	Смысл	Когда выставляется
«1.»	нажато несколько клавиш	опрос клавиатуры нашёл больше одного замкнутого контакта
«2.»	нажатие не распознано	прерывание пришло, а замкнутых контактов нет
«3.»	переполнение буфера	запись X_2 затёрла непрочитанный отсчёт
«4.»	повторный «Тест»	«Тест» нажат, пока идёт строб Y_1

В реализованных фрагментах опрос клавиатуры возвращает 0FFh, если не найдено ровно одно нажатие, а запись в буфер устанавливает F_OVF при вытеснении старого отсчёта. Преобразование этих состояний в индикацию «1.», «2.» или «3.», а также проверка повторного нажатия «Тест» для кода «4.» относятся к обработчику полной системы и показаны в общей схеме алгоритма; отдельные проверочные программы индикатором не управляют.

Сигнал Y_2 выдаётся 8-разрядным кодом в выходной регистр (адрес 0E000h), строб — на выводе P1.6; сигнал Y_1 — на выводе P1.5.

В 16-разрядном режиме (режим 1) таймер 0 перезагружается в обработчике прерывания; задержка входа в прерывание (несколько машинных циклов на тик) компенсируется поправкой к загружаемому значению или учитывается как погрешность длительности. В режиме 2 (8 разрядов с автоперезагрузкой) перезагрузку выполняет сам таймер, а тик набирается программным счётчиком переполнений.

ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ

Таблица 7 — Сокращения

Сокращение	Расшифровка
МК	микроконтроллер
МПС	микропроцессорная система
ОЗУ	оперативное запоминающее устройство
ПЗУ	постоянное запоминающее устройство
ЕСПД	единая система программной документации
ТЗ	техническое задание
CS	Chip Select, выбор микросхемы
IRQ	запрос прерывания
SP	указатель стека
DPTR	16-разрядный регистр-указатель данных
dp	десятичная точка индикатора

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]

ПРИЛОЖЕНИЕ А

Листинги программ

Программа 1 — файл «Иванов ДА-код-1»

```
; Иванов Д.А., А-12-23, 5, v1
$INCLUDE (vars.inc)           ; параметры варианта формирует генератор
DSEG AT 30h
; дополнительные ячейки ОЗУ не нужны: временные значения хранятся в регистрах
CSEG AT 0h
    sjmp START
org 03h                          ; INT0 — нажатие клавиши
    call KbScan ; %proc%
    reti
org 0bh                          ; заглушка Timer0
    nop
    reti
org 13h                          ; заглушка INT1
    nop
    reti
org 1bh                          ; заглушка Timer1
    nop
    reti
org 23h                          ; заглушка UART
    nop
    reti
org 2bh
START:
; Инициализация МК *****
    mov SP, #07h                ; стек начинается после банка регистров 0
    setb PIN_CSEN               ; разрешить дешифратор внешних устройств
    mov DPTR, #ADR_KB           ; адрес регистра столбцов клавиатуры
    clr A                       ; все столбцы держать в нуле для появления INT0
    movx @DPTR, A
    setb IT0                    ; INT0 срабатывает по спаду
    clr IE0                     ; убрать возможный запрос от инициализации
    setb EX0                    ; разрешить прерывание клавиатуры
    setb EA                     ; общее разрешение прерываний
    jmp $ ; %stop%

; KbScan — опрос всех контактов матрицы 3×4 по столбцам.
; Вход: нет. Выход: A — код единственной клавиши по таблице 3 ТЗ, иначе 0FFh.
; Портит: B, DPTR, R0–R5, PSW. Перед возвратом снова опускает все столбцы.
KbScan:
    mov DPTR, #ADR_KB           ; одна ячейка служит для записи столбцов и чтения строк
    mov R0, #1                  ; маска выбранного столбца
    mov R1, #0                  ; число найденных замкнутых контактов
    mov R2, #0FFh               ; код первой найденной клавиши
    mov R3, #0                  ; номер столбца, начиная с нуля
KB_COLUMN:
    mov A, R0                   ; выбранный столбец должен получить нулевой уровень
    cpl A
    anl A, #(1 SHL KB_COLS) - 1
    movx @DPTR, A               ; остальные столбцы временно подняты в единицу
    movx A, @DPTR               ; прочитать активные нулём строки
    anl A, #(1 SHL KB_ROWS) - 1
    cpl A                       ; после инверсии единица означает нажатую клавишу
    anl A, #(1 SHL KB_ROWS) - 1
    mov R5, A                   ; сохранить биты строк текущего столбца
    mov R4, #0                  ; номер проверяемой строки
KB_ROW:
    mov A, R5
    anl A, #1                   ; проверить младший ещё не разобранный бит
    jz KB_NEXT_ROW
    inc R1                      ; учитывается каждое замыкание во всей матрице
    mov A, R1
    cjne A, #1, KB_NEXT_ROW     ; код нужен только для первого замыкания
    mov A, R4
    cjne A, #3, KB_DIGIT        ; последняя строка содержит 0, «Т» и «С»
    mov A, R3
    jz KB_SAVE                  ; Row4/Col1 соответствует цифре 0
    add A, #9                   ; Row4/Col2 и Row4/Col3 дают коды 10 и 11
    sjmp KB_SAVE
```

```

KB_DIGIT:
    mov B, #KB_COLS          ; для первых трёх строк код = row*3+col+1
    mul AB
    add A, R3
    inc A
KB_SAVE:
    mov R2, A                ; запомнить код единственного кандидата
KB_NEXT_ROW:
    mov A, R5                ; перейти к следующему биту строки
    clr C
    rrc A
    mov R5, A
    inc R4
    cjne R4, #KB_ROWS, KB_ROW
    mov A, R0                ; выбрать следующий столбец
    rl A
    mov R0, A
    inc R3
    cjne R3, #KB_COLS, KB_COLUMN
    clr A                    ; вернуть все столбцы в ноль до следующего нажатия
    movx @DPTR, A
    clr IE0                  ; сканирование могло создать лишний фронт запроса
    mov A, R1
    cjne A, #1, KB_BAD       ; ноль или несколько клавиш считаются ошибкой
    mov A, R2
    ret
KB_BAD:
    mov A, #0FFh
    ret
END

```

Программа 2 — файл «Иванов ДА-код-2»

```

; Иванов Д.А., А-12-23, 5, v1
$INCLUDE (vars.inc)          ; параметры варианта формирует генератор
DSEG AT 30h
; дополнительные ячейки ОЗУ не нужны
CSEG AT 0h
    sjmp START
org 03h                        ; заглушка INT0
    nop
    reti
org 0bh                        ; заглушка Timer0
    nop
    reti
org 13h                        ; заглушка INT1
    nop
    reti
org 1bh                        ; заглушка Timer1
    nop
    reti
org 23h                        ; заглушка UART
    nop
    reti
org 2bh
START:
; Инициализация МК *****
    mov SP, #07h              ; стек не пересекается с указателями буфера
    setb PIN_CSEN              ; разрешить дешифратор внешней памяти
    mov DPTR, #BUF_START      ; пустой буфер начинается с первой ячейки кольца
    mov HEAD_L, DPL
    mov HEAD_H, DPH
    mov TAIL_L, DPL
    mov TAIL_H, DPH
    setb F_EMPTY              ; после сброса данных нет
    clr F_OVF
    mov A, #55h               ; тестовый отсчёт для вызова процедуры
    call BufWrite ; %proc%
    jmp $ ; %stop%

; BufWrite — запись байта в кольцевой буфер с политикой полной ёмкости V.
; Вход: A — новый отсчёт. Выход: F_EMPTY = 0; F_OVF = 1 только при вытеснении старого байта.
; Портит: A, B, DPTR, R6, R7, PSW.
BufWrite:
    mov B, A                  ; сохранить отсчёт на время проверки указателей

```

```

    clr F_OVF                ; обычная запись снимает прежний признак переполнения
    jnb F_EMPTY, BW_STORE    ; в пустом кольце совпадение указателей не означает полноту
    mov A, HEAD_L
    cjne A, TAIL_L, BW_STORE
    mov A, HEAD_H
    cjne A, TAIL_H, BW_STORE
    mov DPL, TAIL_L          ; буфер полон: удалить самый старый отсчёт
    mov DPH, TAIL_H
    inc DPTR
    mov R6, DPL              ; сохранить полученный адрес для сравнения с концом
    mov R7, DPH
    mov DPTR, #BUF_END
    mov A, R6
    cjne A, DPL, BW_TAIL_KEEP
    mov A, R7
    cjne A, DPH, BW_TAIL_KEEP
    mov DPTR, #BUF_START     ; после BUF_END кольцо продолжается с начала
    sjmp BW_TAIL_SAVE
BW_TAIL_KEEP:
    mov DPL, R6
    mov DPH, R7
BW_TAIL_SAVE:
    mov TAIL_L, DPL
    mov TAIL_H, DPH
    setb F_OVF               ; новый байт вытеснил самый старый
BW_STORE:
    mov DPL, HEAD_L          ; «голова» указывает на место новой записи
    mov DPH, HEAD_H
    mov A, B
    movx @DPTR, A            ; записать отсчёт левым портом IDT7005
    inc DPTR
    mov R6, DPL              ; проверить необходимость заворота головы
    mov R7, DPH
    mov DPTR, #BUF_END
    mov A, R6
    cjne A, DPL, BW_HEAD_KEEP
    mov A, R7
    cjne A, DPH, BW_HEAD_KEEP
    mov DPTR, #BUF_START
    sjmp BW_HEAD_SAVE
BW_HEAD_KEEP:
    mov DPL, R6
    mov DPH, R7
BW_HEAD_SAVE:
    mov HEAD_L, DPL          ; сохранить адрес следующей свободной ячейки
    mov HEAD_H, DPH
    clr F_EMPTY              ; после записи буфер заведомо не пуст
    ret
END

```

Программа 3 — файл «Иванов ДА-код-3»

```

; Иванов Д.А., А-12-23, 5, v1
$INCLUDE (vars.inc)        ; параметры варианта формирует генератор
DSEG AT 30h
X1:    DS 1                 ; цифра с клавиатуры, ноль задаёт пассивный Y2
X2:    DS 1                 ; очередной отсчёт из кольцевого буфера
CSEG AT 0h
    sjmp START
org 03h                        ; заглушка INT0
    nop
    reti
org 0bh                        ; заглушка Timer0
    nop
    reti
org 13h                        ; заглушка INT1
    nop
    reti
org 1bh                        ; Timer1 — окончание строба Y2
    clr TR1                 ; остановить таймер до следующего вызова
    setb PIN_Y2             ; вернуть строб в пассивную единицу
    clr TF1                 ; снять обработанный запрос переполнения
    reti
org 23h                        ; заглушка UART
    nop

```

```

    reti
org 2bh
START:
; Инициализация МК *****
    mov SP, #07h          ; стек расположен ниже переменных X1 и X2
    setb PIN_CSEN          ; разрешить дешифратор внешних устройств
    setb PIN_Y2            ; начальное состояние stroba пассивное
    mov TMOD, #11h        ; оба таймера настроены на 16-разрядный режим 1
    clr TR1
    clr TF1
    setb ET1                ; разрешить завершение stroba от Timer1
    setb EA
    mov X1, #3             ; тестовые входные данные для вызова процедуры
    mov X2, #200
    call Y2Out ; %proc%
    jmp $ ; %stop%

; Y2Out – вычисление Y2 и запуск сопровождающего stroba Timer1.
; Вход: X1 и X2 в ОЗУ. Выход: байт записан в ADR_Y2, PIN_Y2 удерживается в нуле T2 мкс.
; Портит: A, DPTR, PSW.
Y2Out:
    setb PIN_CSEN          ; гарантировать доступ к регистру Y2
    clr TR1                ; повторный вызов начинает новый полный интервал
    setb PIN_Y2            ; завершить прежний strob, если он ещё выполнялся
    clr TF1
    mov A, X1
    jz Y2_ZERO             ; при X1 = 0 выход должен быть пассивным
    add A, X2
    add A, #Y2_CONST        ; переполнение отбрасывается: результат по mod 256
    sjmp Y2_WRITE
Y2_ZERO:
    clr A
Y2_WRITE:
    mov DPTR, #ADR_Y2
    movx @DPTR, A          ; сначала обновить данные выходного регистра
    mov TH1, #T2_RELOAD_H  ; затем подготовить длительность stroba T2
    mov TL1, #T2_RELOAD_L
    setb TR1
    clr PIN_Y2              ; активный ноль выдаётся только после записи Y2
    ret
END

```